

Version 2.0

NovaCart Account Dashboard — Requirements Document

1. Project Overview

Project: NovaCart Account Dashboard

Client: NovaCart

Prepared by: App Consultant

Purpose: Build a web application that allows NovaCart account managers to explore customers sales data in real time, without relying on the analytics team.

2. Business Objectives

- Eliminate weekly Excel reports that are always at least one week out of date
- Allow account managers across 30+ countries to self-serve sales data from a browser
- Remove dependency on Slack messages to the analytics team for customer performance queries

3. Stakeholders

Role	Responsibility
App Developer	Builds the API, frontend, Docker images, deploy app, technical live demo
App Consultant	Owns requirements, design doc, presentation (business context, tradeoffs, architecture)
Facilitator	NovaCart client contact; handles Snowflake credentials and service deployment
NovaCart Account Managers	End users of the dashboard

4. Architecture Overview

Layer	Description
-------	-------------

Snowflake (Data)	Hosts Gold tables; backend queries directly
FastAPI (Backend)	REST API built in Python; exposes analytics endpoints
React (Frontend)	Browser-based UI with three dashboard views
NGINX router	Route /api/* req to backend and all other req to frontend
Authorization layer	Oauth using JWT
SPCS (Deployment)	Hosts Docker containers; serves the app at a public URL

5. Data Sources The following Snowflake Gold tables are available:

- fact_orders
- dim_customer
- dim_product
- dim_date

6. Functional Requirements

6.1 API Requirements

ID	Requirement
FR-001	The API shall be built in Python 3.11 using FastAPI
FR-002	The API shall expose a /health or root endpoint to confirm it is running
FR-006	All endpoints shall accept query parameters (e.g., date ranges, filters) — no hardcoded values
FR-007	All endpoints shall return paginated or limited results (no full table dumps)
FR-008	All endpoints shall return an empty array [] when no data is found — never null

FR-009	The API shall be fully documented and accessible at /docs via Swagger
FR-010	The API shall return JSON, along with correct http status code
RF-011	The API needs a login page and redirects all unauthorized requests to the login screen

6.2 Frontend Requirements

ID	Requirement
FR-011	The frontend shall be built in React 18
FR-012	The frontend shall include a Customer List — showing individual customer performance data
FR-013	The frontend shall include an Orders Overview — showing order trends and summaries
FR-014	The frontend shall include a Product Performance — showing product-level analytics
FR-015	Each view shall include relevant charts, tables, and filters
FR-016	The frontend shall be connected to the FastAPI backend and display real Snowflake data
FR-017	Loading actions need to show loading state
FR-018	Never show raw http error code -- always handle errors gracefully
FR-019	The frontend shall redirect all unauthorized page access to the login page

6.3 Deployment Requirements

ID	Requirement
FR-020	The application shall be containerized using Docker
FR-021	Docker images shall be built and pushed to the Snowflake image registry
FR-022	The application shall be deployed on SPCS (Snowflake Container Platform Services)
FR-023	The deployed application shall be accessible from any browser via a public URL

7. Non-Functional Requirements

ID	Requirement
NFR-001	API endpoints shall respond within a reasonable time; avoid returning full, unfiltered datasets
NFR-002	Snowflake cursors shall always be closed after use (use try/finally blocks)
NFR-003	The /docs page shall be understandable to anyone — not just the developer who built it
NFR-004	The application must work with real NovaCart data — no mocked or hardcoded responses

8. API Endpoint Specification

EP-001 — Franchise Summary

Field	Detail
Endpoint	GET /franchise/{franchise_id}/summary

Path Parameter	franchise_id (integer) — the unique ID of the franchise
Input Parameters	None
Expected Response	<pre>{ "franchise_id": 1, "total_revenue": 1284750.00, "total_orders": 8432, "active_customers": 1204, "date_range": { "start": "2022-01-01", "end": "2022-10-31" } }</pre>
Acceptance Criteria	Returns a populated JSON object for a valid franchise_id; returns 404 if franchise not found; returns 401 if unauthorized

EP-002 — Franchise Orders

Field	Detail
Endpoint	GET /franchise/{franchise_id}/orders
Path Parameter	franchise_id (integer) — the unique ID of the franchise
Input Parameters	start (YYYY-MM-DD, required), end (YYYY-MM-DD, required)
Expected Response	<pre>[{ "month": "2022-01", "order_count": 842, "revenue": 128450.00 }, { "month": "2022-02", "order_count": 910, "revenue": 141230.00 }]</pre>
Acceptance Criteria	Returns [] when no orders exist in range; returns 400 if date format is invalid; never returns null; returns 401 if unauthorized

EP-003 — Franchise Products

Field	Detail
Endpoint	GET /franchise/{franchise_id}/products
Path Parameter	franchise_id (integer) — the unique ID of the franchise
Input Parameters	start (YYYY-MM-DD, required), end (YYYY-MM-DD, required)
Expected Response	[{ "product_name": "Widget A", "category": "Electronics", "units_sold": 342, "revenue": 24300.00 }, { "product_name": "Gadget B", "category": "Electronics", "units_sold": 290, "revenue": 19850.00 }]
Acceptance Criteria	Returns [] when no products exist in range; returns 400 if date format is invalid; never returns null;

EP-004 — Franchise Customers

Field	Detail
Endpoint	GET /franchise/{franchise_id}/customers
Path Parameter	franchise_id (integer) — the unique ID of the franchise
Input Parameters	start (YYYY-MM-DD, required), end (YYYY-MM-DD, required)
Expected Response	[

	<pre>{ "customer_id": 1001, "first_name": "Ana", "last_name": "García", "country": "Mexico", "total_orders": 14, "total_spent": 4320.00 }, { "customer_id": 1042, "first_name": "John", "last_name": "Smith", "country": "USA", "total_orders": 11, "total_spent": 3890.00 } }</pre>
Acceptance Criteria	Returns [] when no customers exist in range; returns 400 if date format is invalid; never returns null

EP-005 — Franchise Countries

Field	Detail
Endpoint	GET /franchise/{franchise_id}/countries
Path Parameter	franchise_id (integer) — the unique ID of the franchise
Input Parameters	start (YYYY-MM-DD, required), end (YYYY-MM-DD, required)
Expected Response	<pre>[{ "country": "Mexico", "total_orders": 14, "total_revenue": 4320.00 }, { "country": "USA", "total_orders": 11, "total_revenue": 3890.00 }]</pre>

Acceptance Criteria	Returns [] when no countries exist in range; returns 400 if date format is invalid; never returns null
----------------------------	--

9. Frontend Specification

View Name	Purpose	Components Required	Acceptance Criteria
View 1 — Orders Overview	Give account managers a high-level summary of sales activity for the selected date range.	- Stat card: Total Revenue - Stat card: Total Orders - Bar or line chart: Monthly Revenue Over Time - Bar chart: Revenue by Country - Date range filter: Start Date, End Date, Apply button	- Displays total revenue and total orders for the selected date range - Displays monthly revenue trend in a bar or line chart - Displays revenue by country in a bar chart - Date filter updates all values and charts when Apply is clicked - Uses real API data, not mock data
View 2 — Product Performance	Let account managers evaluate which products are performing best during the selected date range.	- Bar chart: Top 10 Products by Revenue - Table:	- Displays top 10 products ranked by revenue - Displays a

		Product Name, Category, Units Sold, Revenue - Date range filter: Start Date, End Date, Apply button	table with product name, category, units sold, and revenue - Date filter updates both chart and table when Apply is clicked - Uses real API data, not mock data
View 3 — Customer List	Let account managers review the highest-value customers for the selected date range.	- Table: Top 20 Customers by Revenue - Columns: Name, Country, Order Count, Total Spent - Sortable rows/columns - Date range filter: Start Date, End Date, Apply button	- Displays top 20 customers by revenue - Table includes name, country, order count, and total spent - Rows can be sorted by any column - Date filter updates the table when Apply is clicked - Uses real API data, not mock data

10. Final Deliverables Checklist

- REST API with 5 working endpoints deployed and documented at /docs
- React frontend with 3 dashboard views connected to real Snowflake data

- Application deployed on SPCS at a public URL
 - Snowpark Container Services (SPCS) authentication manages user and programmatic access through **OAuth login flows** among other things
- Solution Design Document (1–2 pages) written by App Consultant
- 10-minute presentation + 5-minute Q&A ready for the panel
- README updated in the GitHub repo

11. Sign-Off

Name	Role	Status
App Developer	Reviews and confirms requirements before Day 2	Pending
App Consultant	Authors and owns this document	Pending